

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG TP.HCM
KHOA ĐIỆN TỬ VIỄN THÔNG

BÁO CÁO THỰC HÀNH TUẦN 4: MÔ PHỎNG LOGIC BẰNG TRÌNH BIÊN DỊCH VCS & GIAO DIỆN DVE

Họ và tên Sinh viên:	Lê Ngọc Tường
Mã số Sinh viên:	23207124
Lớp học:	23DTV_CLC3
Môn học:	Thực hành thiết kế vi mạch điện tử

1. Thiết kế và kiểm chứng mạch tổ hợp

a) Phân tích lý thuyết

Mạch tổ hợp thực hiện hàm logic:

$$Y = \sim A . \sim B . \sim C + A . \sim B . \sim C + A . \sim B . C$$

Rút gọn biểu thức logic: $Y = \sim B . (\sim C + A)$

Bảng chân trị ngõ ra đối chứng:

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

b) Mã nguồn Verilog

Mạch tổ hợp được thiết kế bằng ngôn ngữ Verilog với hàm logic gán liên tục (assign).

Mã nguồn thiết kế mạch tổ hợp:

```

1 module combinational (
2     input A,
3     input B,
4     input C,
5     output Y
6 );
7     assign Y = (~A & ~B & ~C) | (A & ~B & ~C) | (A & ~B & C);
8 endmodule

```

Testbench chạy mô phỏng với tất cả 8 tổ hợp ngõ vào:

Mã nguồn chạy mô phỏng:

```

1 `timescale 1ns/1ns
2
3 module combinational_tb;
4     reg A, B, C;
5     wire Y;
6
7     combinational uut (
8         .A(A), .B(B), .C(C), .Y(Y)
9     );
10
11     initial begin
12         $monitor($time, " ns | A=%b B=%b C=%b -> Y=%b", A, B, C, Y);
13         A = 0; B = 0; C = 0; #10;
14         A = 0; B = 0; C = 1; #10;
15         A = 0; B = 1; C = 0; #10;
16         A = 0; B = 1; C = 1; #10;
17         A = 1; B = 0; C = 0; #10;
18         A = 1; B = 0; C = 1; #10;
19         A = 1; B = 1; C = 0; #10;
20         A = 1; B = 1; C = 1; #10;
21         $finish;

```

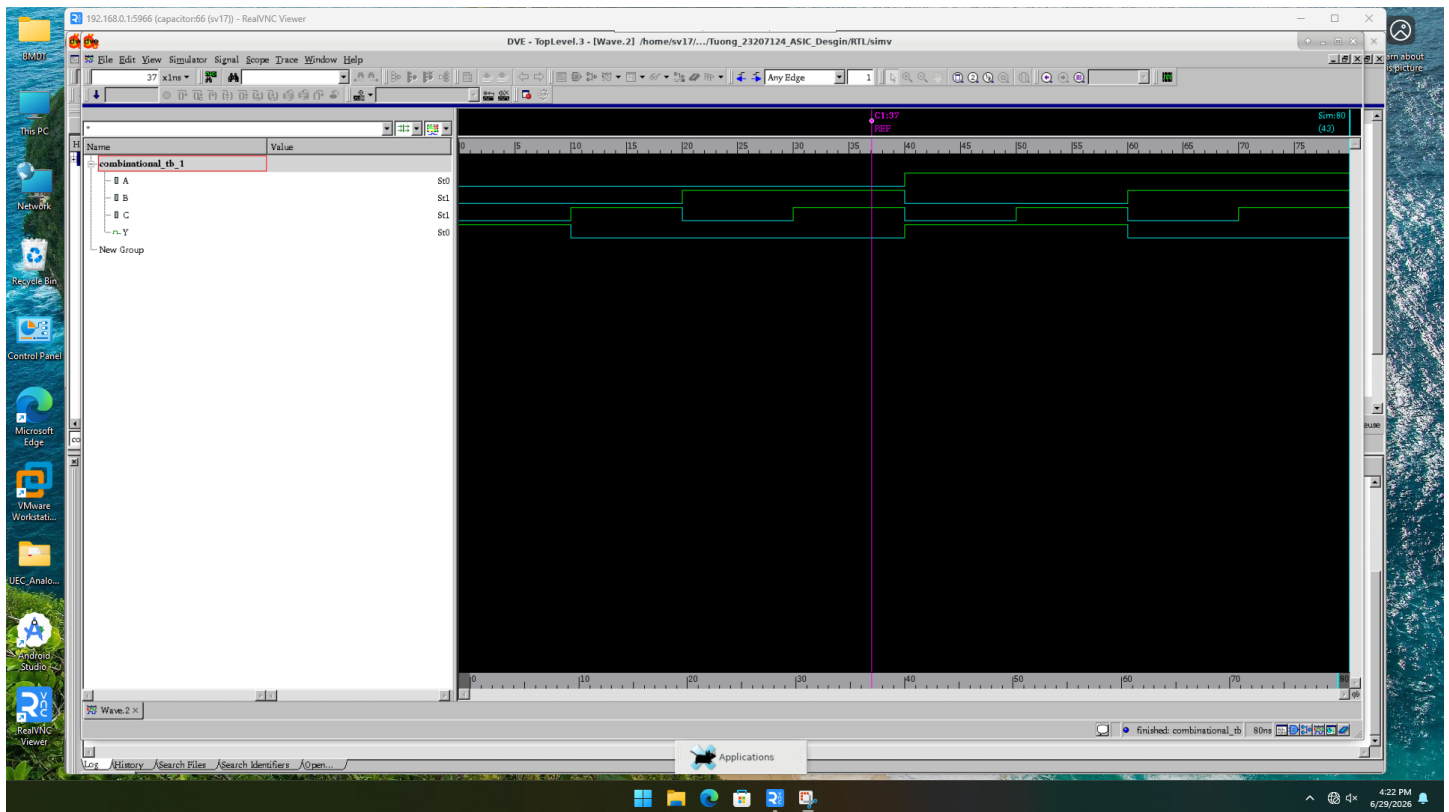
```

22     end
23 endmodule

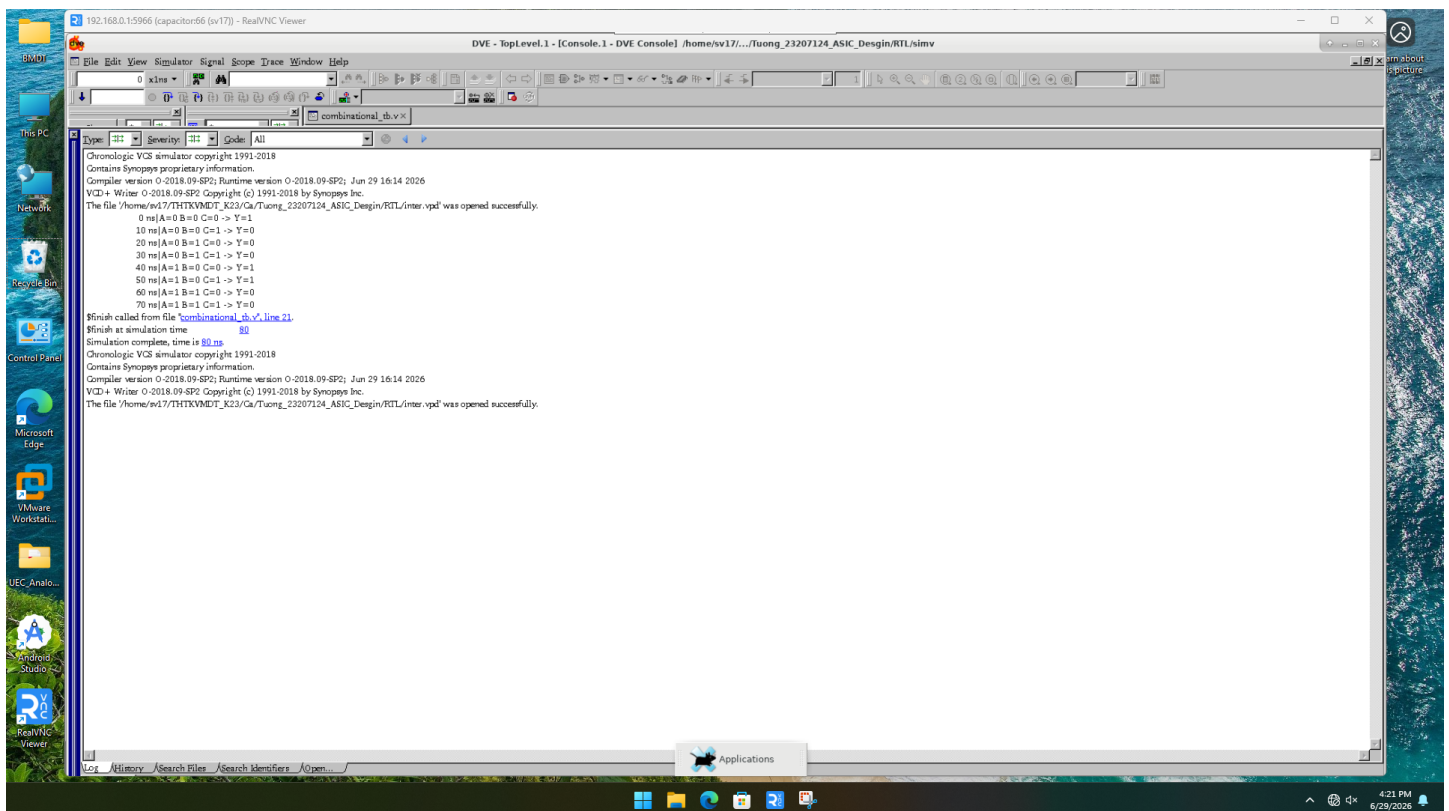
```

c) Kết quả mô phỏng

Giản đồ xung và thông tin xuất ra màn hình mô phỏng:



Giản đồ xung mạch tổ hợp



Kết quả Console Monitor

2. Thiết kế và kiểm chứng mạch đếm 8-bit Counter

a) Phân tích lý thuyết

Mạch đếm tăng dần sau mỗi chu kỳ xung nhịp clock khi chân reset clear ở mức 0. Khi clear lên mức 1, ngõ ra lập tức đưa giá trị về 0 bất chấp trạng thái của clock.

Bộ đếm 8-bit có thể đếm từ 0 đến 255. Mỗi cạnh lên của clock sẽ tăng giá trị hiện tại lên 1 đơn vị. Khi đạt giá trị 255 (11111111), giá trị sẽ quay về 0 (00000000) ở cạnh clock tiếp theo.

b) Mã nguồn Verilog

Bộ đếm được thiết kế với biến reg lưu trạng thái, cập nhật tại cạnh lên của clock:

Mã nguồn thiết kế bộ đếm:

```

1  module counter (
2      input clear,
3      input clock,
4      output reg [7:0] state
5  );
6      always @(posedge clock or posedge clear) begin
7          if (clear)
8              state <= 8'b0000_0000;
9          else
10             state <= state + 1'b1;
11      end
12  endmodule

```

Testbench kiểm tra hoạt động reset và đếm:

Mã nguồn chạy mô phỏng:

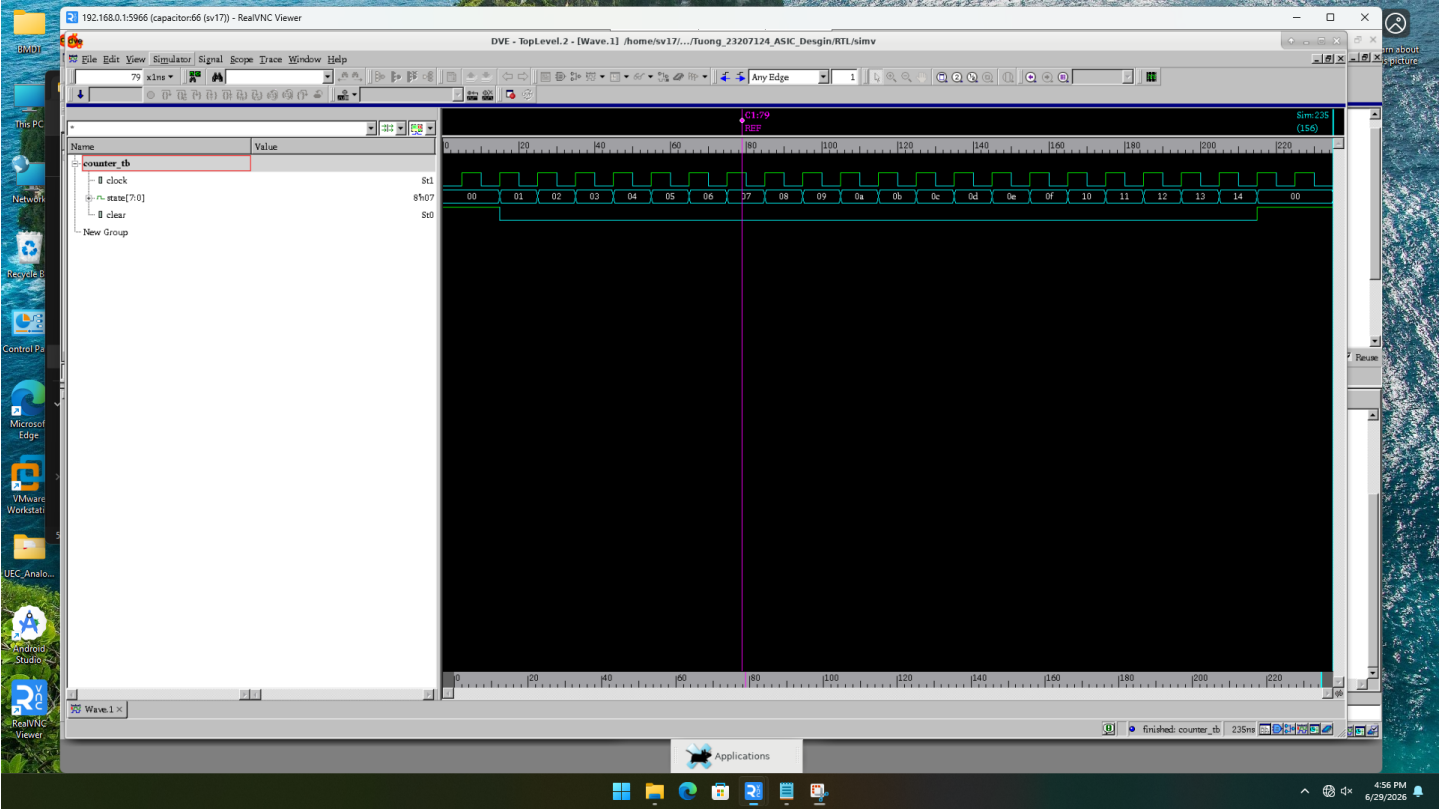
```

1  `timescale 1ns/1ns
2
3  module counter_tb;
4      reg clear;
5      reg clock;
6      wire [7:0] state;
7
8      counter uut (
9          .clear(clear),
10         .clock(clock),
11         .state(state)
12     );
13
14     always #5 clock = ~clock;
15
16     initial begin
17         clock = 0;
18         clear = 1;
19         #15;
20         clear = 0;
21         #200;
22         clear = 1;
23         #20;
24         $finish;
25     end
26
27     initial begin
28         $monitor($time, " ns | clear=%b | state = %d (bin: %b)", clear, state, state);
29     end
30 endmodule

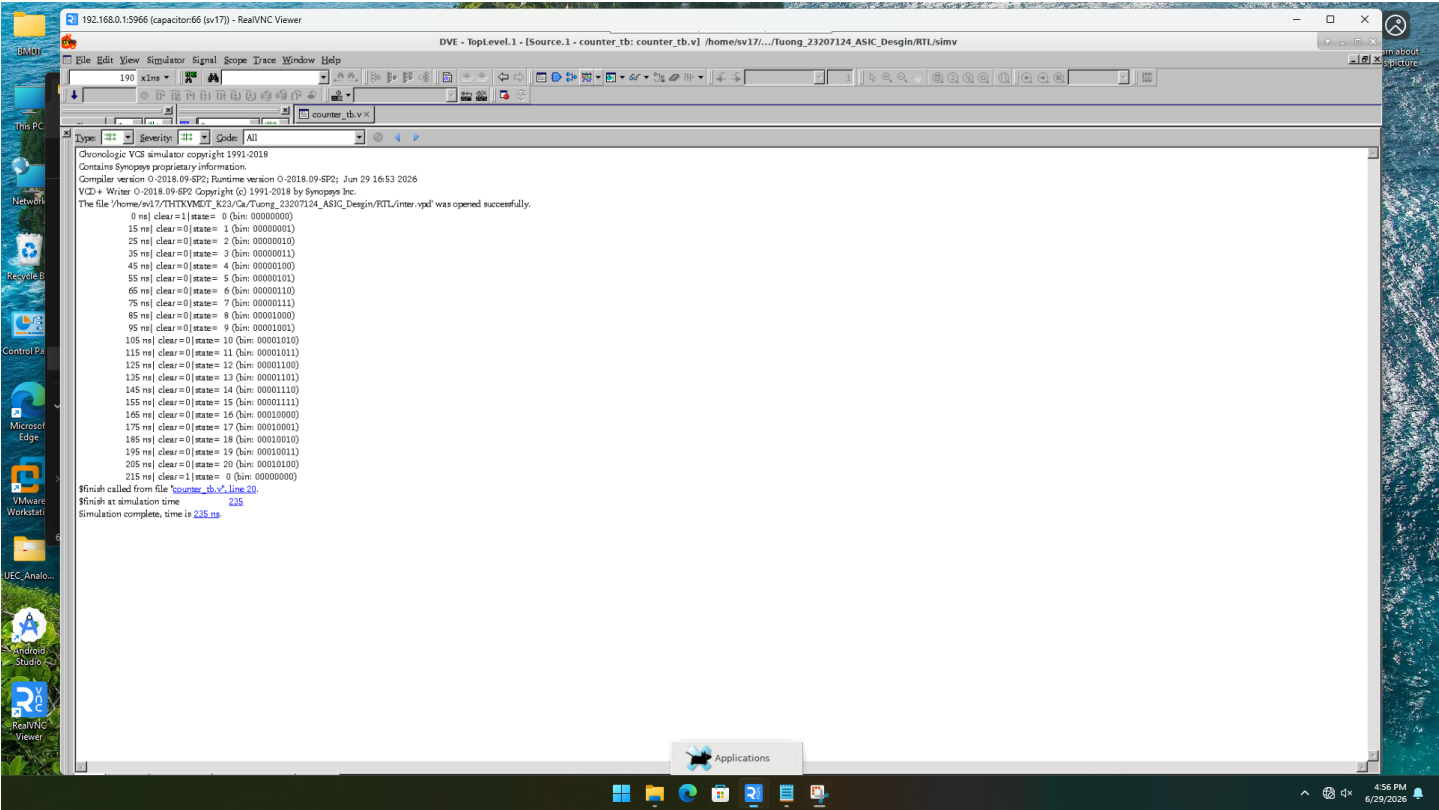
```

c) Kết quả mô phỏng

Giản đồ xung và thông tin xuất ra màn hình mô phỏng:



Giản đồ xung mạch đếm Counter



Kết quả Console Monitor

3. Decoder 3-to-8 và MUX 8-to-1

a) Decoder 3-to-8 có chân Enable

Decoder 3-to-8 nhận đầu vào 3-bit và kích hoạt 1 trong 8 ngõ ra tương ứng. Khi Enable ở mức 0, toàn bộ ngõ ra được đặt về 0. Khi Enable ở mức 1, một trong 8 ngõ ra sẽ được kích hoạt tùy theo giá trị đầu vào.

Bảng chân trị Decoder 3-to-8:

in[2]	in[1]	in[0]	en	out[7:0]
0	0	0	1	0000_0001
0	0	1	1	0000_0010
0	1	0	1	0000_0100
0	1	1	1	0000_1000
1	0	0	1	0001_0000
1	0	1	1	0010_0000
1	1	0	1	0100_0000
1	1	1	1	1000_0000

b) Mã nguồn Verilog - Decoder 3-to-8

Decoder được thiết kế bằng cấu trúc case để giải mã 3-bit đầu vào:

Mã nguồn Decoder 3-to-8:

```

1  module decoder3to8 (
2      input [2:0] in,
3      input en,
4      output reg [7:0] out
5  );
6      always @(*) begin
7          if (!en)
8              out = 8'b0000_0000;
9          else begin
10             case (in)
11                 3'b000: out = 8'b0000_0001;
12                 3'b001: out = 8'b0000_0010;
13                 3'b010: out = 8'b0000_0100;
14                 3'b011: out = 8'b0000_1000;
15                 3'b100: out = 8'b0001_0000;
16                 3'b101: out = 8'b0010_0000;
17                 3'b110: out = 8'b0100_0000;
18                 3'b111: out = 8'b1000_0000;
19                 default: out = 8'b0000_0000;
20             endcase
21         end
22     end
23 endmodule

```

c) MUX 8-to-1 cấu trúc từ MUX 2-to-1

MUX 8-to-1 được xây dựng từ 3 tầng MUX 2-to-1. Mỗi tầng giảm số lượng tín hiệu đi một nửa cho đến khi còn 1 ngõ ra duy nhất. Đầu vào select 3-bit chọn 1 trong 8 tín hiệu đầu vào.

Mạch thành phần MUX 2-to-1:

```

1  module mux2to1 (
2      input a, b, sel,
3      output out
4  );
5      assign out = sel ? b : a;

```

```
6 endmodule
```

Mạch chính MUX 8-to-1:

```
1 module mux8to1 (
2     input [7:0] in,
3     input [2:0] sel,
4     output out
5 );
6     wire [3:0] w1;
7     wire [2:0] w2;
8
9     mux2to1 m10 (in[0], in[1], sel[0], w1[0]);
10    mux2to1 m11 (in[2], in[3], sel[0], w1[1]);
11    mux2to1 m12 (in[4], in[5], sel[0], w1[2]);
12    mux2to1 m13 (in[6], in[7], sel[0], w1[3]);
13
14    mux2to1 m20 (w1[0], w1[1], sel[1], w2[0]);
15    mux2to1 m21 (w1[2], w1[3], sel[1], w2[1]);
16
17    mux2to1 m30 (w2[0], w2[1], sel[2], out);
18 endmodule
```

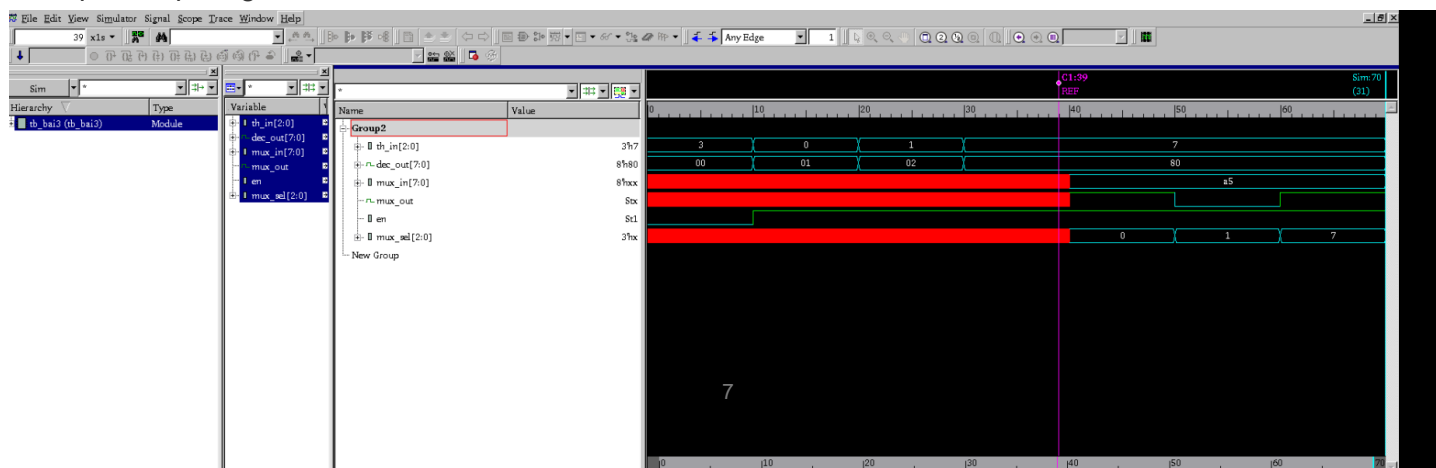
d) Kết quả mô phỏng

Testbench kiểm tra Decoder 3-to-8 với Enable và MUX 8-to-1:

Mã source testbench:

```
1 `timescale 1ns/1ps
2
3 module tb_bai3;
4     reg [2:0] th_in;
5     reg en;
6     wire [7:0] dec_out;
7     reg [7:0] mux_in;
8     reg [2:0] mux_sel;
9     wire mux_out;
10
11    decoder3to8 uut_dec (.in(th_in), .en(en), .out(dec_out));
12    mux8to1 uut_mux (.in(mux_in), .sel(mux_sel), .out(mux_out));
13
14    initial begin
15        en = 0; th_in = 3'b011; #10;
16        en = 1; th_in = 3'b000; #10;
17        th_in = 3'b011; #10;
18        th_in = 3'b111; #10;
19        mux_in = 8'b1010_0101; mux_sel = 3'b000; #10;
20        mux_sel = 3'b001; #10;
21        mux_sel = 3'b111; #10;
22        $finish;
23    end
24 endmodule
```

Kết quả mô phỏng:



4. Counter 4-bit Up/Down với Parallel Load

a) Phân tích lý thuyết

Bộ đếm 4-bit có khả năng đếm lên hoặc xuống tùy theo tín hiệu mode. Khi load được kích hoạt, giá trị data_in sẽ được nạp vào bộ đếm. Reset bất đồng bộ khi rst_n ở mức 0.

Chức năng các chân:

- rst_n: Reset bất đồng bộ, khi ở mức 0 sẽ đưa bộ đếm về 0
- clk: Xung nhịp clock, bộ đếm cập nhật tại cạnh lên
- load: Chức năng nạp song song, khi load=1 sẽ nạp giá trị từ data_in
- mode: Chọn chế độ đếm, mode=1 đếm lên, mode=0 đếm xuống
- data_in[3:0]: Dữ kiện 4-bit đầu vào khi nạp song song
- count[3:0]: Ngõ ra 4-bit hiển thị giá trị đếm hiện tại

b) Mã nguồn Verilog

Bộ đếm được thiết kế với biến reg lưu trạng thái, cập nhật tại cạnh lên của clock:

Mã nguồn thiết kế bộ đếm Up/Down:

```

1  module counter_updown (
2      input clk,
3      input rst_n,
4      input load,
5      input [3:0] data_in,
6      input mode,
7      output reg [3:0] count
8  );
9      always @(posedge clk or negedge rst_n) begin
10         if (!rst_n)
11             count <= 4'b0000;
12         else if (load)
13             count <= data_in;
14         else begin
15             if (mode)
16                 count <= count + 1'b1;
17             else
18                 count <= count - 1'b1;
19         end
20     end
21 endmodule

```

c) Kết quả mô phỏng

Testbench kiểm tra các chức năng reset, nạp song song, đếm lên và đếm xuống:

Mã source testbench:

```

1  `timescale 1ns/1ps
2
3  module tb_counter;
4      reg clk;
5      reg rst_n;
6      reg load;
7      reg mode;
8      reg [3:0] data_in;
9      wire [3:0] count;
10
11     counter_updown uut (
12         .clk(clk),
13         .rst_n(rst_n),
14         .load(load),

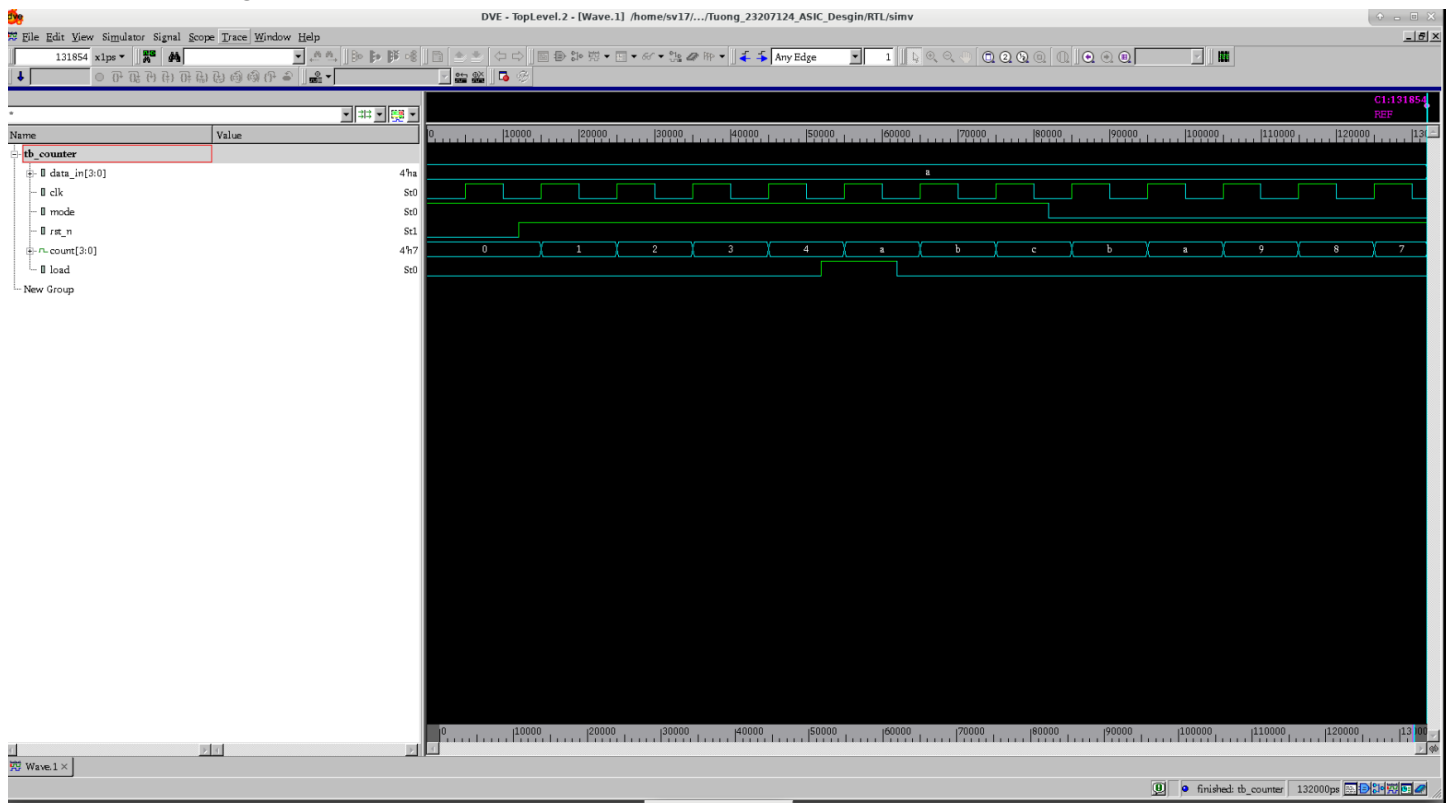
```

```

15     .data_in(data_in),
16     .mode(mode),
17     .count(count)
18 );
19
20 always #5 clk = ~clk;
21
22 initial begin
23     clk = 0; rst_n = 0; load = 0;
24     mode = 1; data_in = 4'b1010;
25     #12 rst_n = 1;
26     #20 mode = 1;
27     #20 load = 1;
28     #10 load = 0;
29     #20 mode = 0;
30     #50 $finish;
31 end
32 endmodule

```

Kết quả mô phỏng:



Kết quả mô phỏng Counter 4-bit Up/Down

5. Accumulator - Bộ cộng tích lũy 8-bit

a) Phân tích lý thuyết

Bộ cộng tích lũy thực hiện phép cộng liên tục giữa giá trị hiện tại `acc_out` và đầu vào `data_in`. Khi `clear` ở mức 1, ngõ ra được reset về 0. Kết quả tích lũy được cập nhật tại mỗi cạnh lên của xung clock.

Công thức tích lũy: $acc_out[n] = acc_out[n-1] + data_in$

Bộ đếm tích lũy được sử dụng phổ biến trong các mạch xử lý tín hiệu số, đặc biệt là trong bộ lọc FIR và phép tính tích chập.

b) Mã nguồn Verilog

Bộ cộng tích lũy được thiết kế với biến `reg` lưu trạng thái:

Mã nguồn thiết kế Accumulator:

```
1 module accumulator (
2     input clk,
3     input clear,
4     input [8:0] data_in,
5     output reg [8:0] acc_out
6 );
7     always @(posedge clk) begin
8         if (clear)
9             acc_out <= 8'b0000_0000;
10        else
11            acc_out <= acc_out + data_in;
12        end
13    endmodule
```

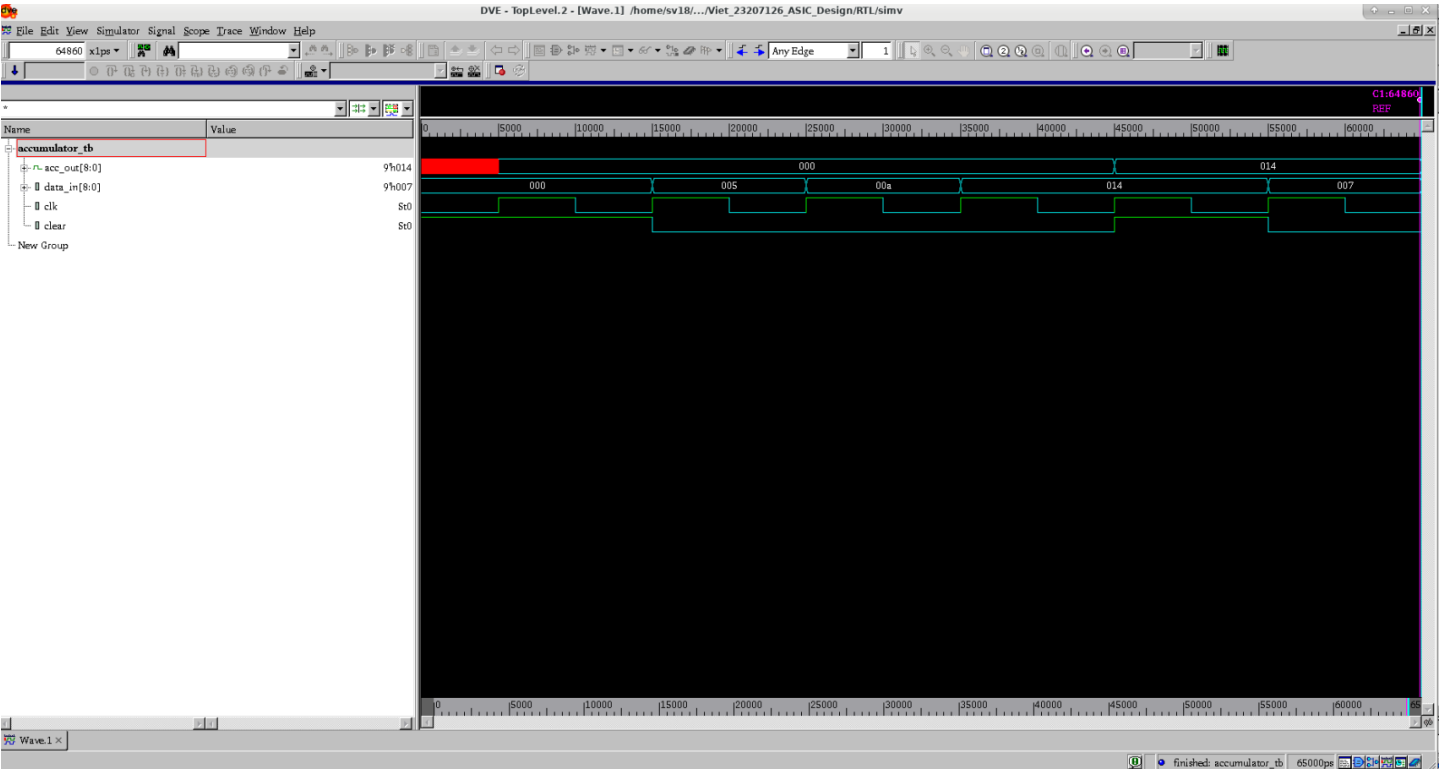
c) Kết quả mô phỏng

Testbench kiểm tra chức năng reset và tích lũy:

Mã source testbench:

```
1 `timescale 1ns/1ps
2
3 module tb_accumulator;
4     reg clk;
5     reg clear;
6     reg [8:0] data_in;
7     wire [8:0] acc_out;
8
9     accumulator uut (
10        .clk(clk),
11        .clear(clear),
12        .data_in(data_in),
13        .acc_out(acc_out)
14    );
15
16    always #5 clk = ~clk;
17
18    initial begin
19        clk = 0; clear = 1; data_in = 8'd0;
20        #15 clear = 0;
21        data_in = 8'd5; #10;
22        data_in = 8'd10; #10;
23        data_in = 8'd20; #10;
24        clear = 1; #10;
25        clear = 0; data_in = 8'd7; #10;
26        $finish;
27    end
28 endmodule
```

Kết quả mô phỏng:



Kết quả mô phỏng Accumulator